

COP4934 Senior Design 1 Scrum & Meetings

Contents

1	Introduction	2
1.1	Sprint Planning	2
1.2	Sprint Review	2
1.3	Sprint Retrospective	2
2	Meeting Minutes	2
2.1	2-16-2026	2
2.2	2-23-2026	3
2.3	3-2-2026	4
2.4	3-9-2026	4
3	Sprint 0 : 2/1/2026 - 2/15/2026	4
3.1	Goal	4
3.2	Sprint Planning	4
3.3	Sprint Review	4
3.4	Sprint Retrospective	5
4	Sprint 1 : 2/15/2026 - 3/1/2026	5
4.1	Goal	5
4.2	Sprint Planning	5
4.3	Sprint Review	6
4.4	Sprint Retrospective	7
5	Sprint 2 : 3/1/2026 - 3/15/2026	7
5.1	Goal	7
5.2	Sprint Planning	7
5.3	Sprint Review	8
5.4	Sprint Retrospective	8
6	Sprint 3 : 3/15/2026 - 3/29/2026	9
6.1	Goal	9
6.2	Sprint Planning	9
6.3	Sprint Review	10
6.4	Sprint Retrospective	10
7	Sprint 5 : 4/12/2026 - 4/26/2026	10
7.1	Goal	10
7.2	Sprint Planning	10
7.3	Sprint Review	10
7.4	Sprint Retrospective	11

1 Introduction

1.1 Sprint Planning

Sprint Planning is an agile event where the scrum team defines the work to be completed during the upcoming sprint. The team establishes a clear sprint goal, selects high-priority items from the product backlog, and breaks them into technical tasks to form the Sprint Backlog. This meeting aligns the team on what can be delivered by the end of the sprint and how the work will be accomplished.

- **Focus:** Planning the upcoming sprint and defining the sprint goal
- **Goal:** Decide what work will be done in the sprint and how it will be completed.
- **Participants:** Scrum Team (Developers, Product Owner, Scrum Master)
- **Outcome:** Sprint Goal, Sprint Backlog, and a shared understanding of the planned work

1.2 Sprint Review

Sprint Review is held at the end of a sprint to evaluate the completed product increment. The scrum team demonstrates finished work, gathers feedback from stakeholders, and discusses progress toward the overall product goals. The outcome of the review is an updated product backlog and a shared understanding of what was accomplished and what should be prioritized next.

- **Focus:** The Product Increment (what was built)
- **Goal:** Inspect the increment, get stakeholder feedback, and adapt the Product Backlog
- **Participants:** Scrum Team (Developers, PO, SM) + Stakeholders.
- **Outcome:** Updated Product Backlog, shared understanding of progress, feedback for future work

1.3 Sprint Retrospective

Sprint Retrospective is an internal scrum team meeting focused on improving the team's development process. The team reflects on what went well, what challenges were encountered, and how collaboration, tools, or workflows can be improved. The result is a set of actionable improvements that the team commits to applying in the next sprint.

- **Focus:** The Process (how the team works)
- **Goal:** Inspect the team's people, processes, and tools; identify improvements
- **Participants:** Scrum Team only (internal event)
- **Outcome:** Actionable improvements to the team's process/collaboration for the next sprint

2 Meeting Minutes

2.1 2-16-2026

Attendance: Everyone in the team attended the meeting.

During this meeting, we assigned the tasks for sprint 1. In the span of the next two weeks, each role will have their own assigned tasks dedicated for their roles, but we will work towards the overall objective of having a basic skeleton or backbone (low implementation) of our project set up and aim to have it dockerized and able to run on everyone's computer. Went through the Jira and added all the tasks and assigned them. Everyone will be in charge of updating their progress of each task on Jira. Below are the tasks that were assigned to each role:

- Cybersecurity/Honeypot Lead:
 - Choose honeypot method: decide on cowrite or custom SSH
 - Deploy first honeypot logging the user, pass, commands, and IP
 - Define threat boundaries: what attacker can do, what they cannot do, and what is logged
 - Export logs as JSON or CSV with one file per session
 - **Hard Deliverable:** A running SSH honeypot that logs real attacker sessions.
- Backend Systems Engineer:
 - Scaffold backend: Fast API, routes, models, services
 - Create API: POST, GET
 - Parse incoming honeypot logs: have framework to implement once backend is fully setup
 - Dockerize backend
 - **Hard Deliverable:** Backend service that accepts attack logs via HTTP.
- Database Engineer:
 - Design schema tables: sessions, events, credentials
 - Implement SQL init script
 - Connect backend to accept inserts and test queries
 - Write sample analytic queries
 - **Hard Deliverable:** Postgres with real tables storing attack data.
- Frontend Visualization Engineer:
 - Scaffold React App: layout, navigation
 - Create Views: sessions table and detail page
 - Connect backend: fetch and render data
 - Dockerize frontend
 - **Hard Deliverable:** Dashboard that displays real or mock sessions.
- ML Threat Analysis:
 - Define ML Problem: clustering or anomaly detection
 - Design features: session duration, command count, password entropy
 - Write pipeline: load data, compute features, plot distributions
 - **Hard Deliverable:** Jupyter notebook that processes attack data.

2.2 2-23-2026

Attendance: Everyone was present during this meeting.

Progress Update:

The team discussed individual progress on assigned roles and tasks. Overall, everyone has made significant progress and actively working towards completion on the current sprint's tasks. We are all consistently updating the Github and Jira as tasks are completed.

The backend is fully dockerized and we confirmed that it is running locally on everyone's machines/ The frontend is currently in progress and will be completed by the end of the week.

The team also reviewed requirements for the Design Proposal and Team Contract document that is due this weekend as well as what each of us needs to fill out.

Scheduling Update: There is a slight scheduling conflict of our meeting times since one of our members has a class during this time, so we agreed to moving our weekly meetings to Mondays at 1:30pm.

2.3 3-2-2026

Attendance: Everyone was present during this meeting.

Progress Update: Everything for sprint 1 was completed. First honeypot was deployed on Cowrie and it can send logs to the backend. We will research further on how to automate this in the future. This current sprint's tasks will be sent after the instructor check ins this week. We will update the Jira with the new tasks and assign them to their correct roles. We will also need to work on the Early status report that is due this weekend and provide MVP, Jira updates, sprint plannings, and Agile event info.

2.4 3-9-2026

Attendance: Everyone was present during this meeting.

Progress Update: In this meeting, we talked about the tasks for the next two sprints since they are connected. The main goal is to move Sentinel Grid to a real cloud honey net with a working dashboard and structured attack data. Zach sent out tasks for everyone for week 4 to 7 and we also updated the Jira board sprints and assigned tasks per role. Yama will start on the template for the Preliminary Design document so we can split up everyone's parts. We will work parallel to efficiently get tasks done. Next week is spring break, so we will just have a basic progress update on implementation and documentation.

3 Sprint 0 : 2/1/2026 - 2/15/2026

3.1 Goal

Our goal for Sprint 0 is to set the foundation of our project and to establish a clear semester plan.

3.2 Sprint Planning

The focus of Sprint 0 is to plan the foundation required to establish a clear semester plan. Our team talked about our skills, preferred programming languages, experience, and what we would each like to do in order to figure out everyone's roles and responsibilities. This will help us figure out what each member needs to develop and learn this semester and clear up any questions before we start implementation.

- What can we deliver by the end of this sprint?
By the end of this sprint, we will deliver a draft of our semester plan that clearly defines team roles, responsibilities, learning gaps, and requirements of the project.
- How will we build it?
We will have a meeting to collaboratively develop the semester plan including planning weekly meetings. Responsibilities will be assigned based on individual strengths and learning goals, and the plan will be documented and reviewed to ensure alignment with project objectives.
- What must be done vs nice:
Must be done: defining team roles, assigning responsibilities, identifying required skills and knowledge
Nice to have: detailed stretch goals and general requirements

3.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
Weekly meeting times were agreed upon to take place online every Mondays at 12pm and in person meetings will take place as needed. We defined team roles, assigned responsibilities, each member had an idea of required skills and tools they needed to learn.

- What worked well:
Communication about everyone's background and skills helped the team assign roles effectively. Collaboration during planning ensured that everyone felt heard and aligned with the project goals and roles.
- Feedback received:
It's important to have certain milestones and deliverables for each sprint. It was also suggested that documentation should be kept updated throughout the semester for each member so we can keep track of what everyone is working on.
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
Added tasks for researching required technologies, setting up the GitHub, and creating documentation repository.

3.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
The team established a clear direction for the semester and ensured everyone understood their responsibilities. Meetings were productive and stayed focused on deliverables and what each person had to do next.
- What challenges or obstacles were encountered:
Aligning everyone's schedules for the weekly meetings and there was a learning curve when it came to learning how to use and understand Jira
- What could be improved in future sprints: The team should also break large goals into smaller, more manageable tasks to improve tracking and accountability. Each member is able to create their own sub tasks and other things they are working on and add them to the Jira and update them as they complete them
- Action items to implement in the next sprint:
Set up the development environment, work on project requirements, assign tasks for the next spring

4 Sprint 1 : 2/15/2026 - 3/1/2026

4.1 Goal

Our goal for Sprint 1 is to complete the design proposal and team contract with the focus on developing the skills and knowledge required for each team role while working on each respective assigned tasks.

4.2 Sprint Planning

During the 2/16/2026 meeting, all team members were present and Sprint 1 tasks were assigned. Each team member received role specific responsibilities to work on building a basic skeleton and low implementation of the project. The team aims to have a functional backbone of the system set up, dockerized, and capable of running on all team members' machines by the end of the sprint. All tasks were added to Jira, and each member is responsible for updating their task progress throughout the sprint.

- What can we deliver by the end of this sprint?
 - A running SSH honeypot that logs attacker sessions (credentials, commands, IP)
 - A dockerized backend service that accepts attack logs
 - A Postgres database with implemented schema tables (sessions, events, credentials) storing attack data

- A dockerized React frontend that displays real or mock session data
- A Jupyter notebook that processes attack data for clustering or anomaly detection
- A basic, integrated project skeleton that runs locally on all team members' machines
- How will we build it?
Each role will focus on completing its assigned deliverables
- What must be done vs nice:
Must be done:
 - Dockerize backend and frontend
 - Ensure the system skeleton runs locally for everyone
 - Complete the design proposal and team contract.

Nice to have:

- Complete all hard deliverables for each role
- Initial integration testing between all components
- Basic analytics visualizations beyond required views

4.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
 - Design proposal and team contract was completed
 - Backend and data base set up and dockerized
 - Front end scaffold completed
 - ML notebooks completed for data analysis and data preprocessing pipeline
 - Docker for backend verified to run successfully across everyone's environments locally
 - Deployed first honeypot on cowrie that can be compromised via ssh root login with no proper authentication
- What worked well:
 - Separate github branches and roles allowed for parallel development across backend, frontend, database, and ml
 - Weekly meetings for updates
 - Regular communication and working through issues together on Discord
 - Jira task tracking for progress and accountability
- Feedback received:
 - Clarity and good documentation needed for other members to know how to run certain parts and ensure reproducibility
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):
 - Built off of this sprint to add the next steps and tasks for the next sprint
 - Idea of exploring more possibilities of supervised learning for classifications

4.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
 - Every member successfully worked on their roles and tasks while learning the technologies they will be using
 - We have established a project skeleton earlier than expected
 - Communication remained consistent
- What challenges or obstacles were encountered:
 - Some of us had to learn how to use docker for the first time
 - Limited availability of realistic labeled attack data cause restriction in experimenting with supervised classification- will need to find another way
- What could be improved in future sprints:
 - Aim to complete assignments the day before so we all can review the day of submissions
 - Improved documentation updates on Github for readmes for different parts of the project structure
- Action items to implement in the next sprint:
 - Look into supervised learning for classification
 - Look into how to connect all the different parts together into a functioning structure (connect back end to ml, to front end, to data, etc.)
 - Not specifically for next sprint but will need to look into automatic sending logs from honeypot to backend in the future

5 Sprint 2 : 3/1/2026 - 3/15/2026

5.1 Goal

Establish the core cloud infrastructure for SentinelGrid so we can move beyond a local Docker only setup. By the end of Sprint 2, the team aims to have the backend prepared for cloud deployment, the database configured for real log storage, at least one honeypot connected to the backend log pipeline, and the frontend prepared for integration with live API data.

changed meeting times to mondays at 1:30

5.2 Sprint Planning

Sprint 2 focuses on turning SentinelGrid from a local prototype into the first version of a cloud-connected cybersecurity monitoring platform. The priority is to deploy or prepare the backend and database in the cloud, confirm that honeypot logs can be accepted and stored through the API, and ensure the frontend can begin consuming real backend data. Since this is the first infrastructure sprint of the new phase, the emphasis is on reliability, connectivity, and end-to-end data flow rather than advanced analytics or polished UI features.

- What can we deliver by the end of this sprint?
 - A cloud-hosted backend deployment or a fully prepared backend deployment environment on a VPS
 - A PostgreSQL database instance configured for SentinelGrid and connected

- A verified log ingestion pipeline where honeypot events can be sent to the backend and stored in the database
- At least one deployed honeypot node producing structured logs
- Backend API endpoints tested for basic functionality such as log submission and session retrieval
- A frontend/dashboard environment connected to backend APIs or prepared for deployment and live integration
- Updated documentation for deployment, infrastructure setup, and ingestion workflow
- Preprocessing data pipeline for ML and feature extraction
- How will we build it? We will work and build each part of the structures in parallel on our own branches on Github. Those relying on other parts needing to be setup first will conduct research on what they need to do/start with a skeleton baseline
- What must be done vs nice:
 - Must be done:**
 - Set up cloud infrastructure for backend deployment
 - Deploy or finalize deployment-ready FastAPI backend
 - Set up PostgreSQL database and confirm schema creation
 - Connect backend to database successfully
 - Insert incoming logs into database correctly
 - Deploy at least one
 - Fix frontend integration with backend APIs
 - Document setup steps and team workflow
 - ML pipelines for data and features

Nice to have:

- Add automatic log forwarding scripts instead of manual testing
- Add input validation and stronger API error handling
- Start preprocessing modularization for ML pipeline integration
- Prepare early visualizations for attack sessions if enough real logs are available

5.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
- What worked well:
- Feedback received:
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):

5.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
- What challenges or obstacles were encountered:
- What could be improved in future sprints:
- Action items to implement in the next sprint:

6 Sprint 3 : 3/15/2026 - 3/29/2026

6.1 Goal

Expand SentinelGrid from a cloud-connected prototype into a more complete live cybersecurity monitoring platform. By the end of Sprint 3, the team aims to stabilize the deployed infrastructure, support multiple honeypot nodes, improve structured log storage and analytics queries, complete the main dashboard views, and prepare the ML pipeline for anomaly detection, clustering, and backend integration.

6.2 Sprint Planning

Sprint 3 focuses on system completion, analytics, and stability. After establishing the cloud deployment and initial log-ingestion pipeline in Sprint 2, this sprint will extend SentinelGrid into a more functional honeynet platform with multiple honeypots, improved database performance, new backend analytics endpoints, and a more complete React dashboard. In parallel, the ML/data pipeline will move from preprocessing and baseline feature extraction toward clustering and anomaly scoring so that attack sessions can later be surfaced as higher-level patterns rather than raw logs alone. ML will also explore the possibility of labeling data for supervised classification

- What can we deliver by the end of this sprint?
 - Two deployed honeypot nodes, including one SSH honeypot and one HTTP honeypot
 - Automatic log forwarding from honeypots into the backend API
 - A stable PostgreSQL database storing structured live attack data
 - Improved database schema and indexes for faster log queries
 - Analytics endpoints for top attacker IPs, attack rates, commands, and session summaries
 - A working React dashboard with pages such as Live Attack Feed, Session Explorer, and Attack Analytics
 - Visualizations for attack frequency, common commands, and top sources of activity
 - Initial clustering and anomaly detection outputs from honeypot sessions and external datasets
 - Documentation for infrastructure, API usage, database maintenance, and ML integration
- How will we build it? We will continue to work in parallel on our own branches on Github. Those relying on other parts needing to be setup first will conduct research on what they need to do/start with a skeleton baseline
- What must be done vs nice:
Must be done:
 - Deploy and maintain both honeypots
 - Ensure automatic log forwarding works
 - Improve raw log schema and add indexes on important fields such as timestamp, IP, and session ID
 - Implement backend analytics endpoints
 - Complete core dashboard pages for live logs, sessions, and analytics
 - Add charts for attack trends and attacker activity
 - Run clustering or anomaly detection pipeline on collected session data
 - Define backend ready output format for anomaly or behavior analysis results

Nice to have:

- Add automatic database backups and export scripts

- Create fake attacker interactions such as fake admin logins or fake filesystem responses
- Add auto refresh and improved filtering in the dashboard
- Add richer session drill-down features in the UI
- Tune anomaly scoring for near real time detection
- Integrate early anomaly scores directly into dashboard views
- Improve UI responsiveness and polish for demo readiness
- Possible labeling subset for ML supervised classification

6.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
- What worked well:
- Feedback received:
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):

6.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
- What challenges or obstacles were encountered:
- What could be improved in future sprints:
- Action items to implement in the next sprint:

7 Sprint 5 : 4/12/2026 - 4/26/2026

7.1 Goal

...

7.2 Sprint Planning

[Description/Summary...]

- What can we deliver by the end of this sprint?
- How will we build it?
- What must be done vs nice:
Must be done:
Nice to have:

7.3 Sprint Review

The team reviewed the following:

- What was completed during the sprint:
- What worked well:
- Feedback received:
- Updates made to the product backlog (new items, changes in scope, or priority adjustments):

7.4 Sprint Retrospective

The team discussed:

- What went well during the sprint:
- What challenges or obstacles were encountered:
- What could be improved in future sprints:
- Action items to implement in the next sprint: